

プログラミングI 演習

Computer Programming I: Exercises

第3回 条件文

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

2

本日の目標

- 条件分岐 (if文, if～else文) ができるようになる
- 簡単な流れ図 (フローチャート) が読めるようになる
- 関係演算子と論理演算子を覚える

本日の資料

1. 前回のおさらい
2. 条件分岐
3. 演習課題

•変数

数値を格納するための変数という概念を知った
int, float, double型変数の利用方法を身につけた

•入出力

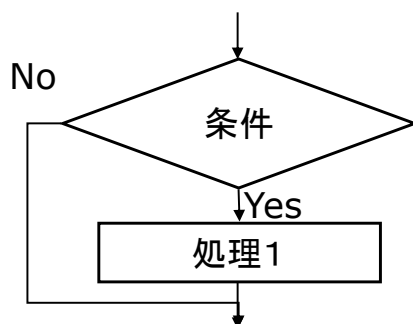
printf: 数値や文字の画面出力の方法
scanf: キーボードからの数値入力の方法
上記の2つを身につけた

2. 条件分岐

if文 ある条件に応じて処理を分岐させる制御構文

条件を満たす場合のみ
{ }(ブロック)を実行

流れ図/Flow Chart



```
if (条件文)
{
    処理1;
}
```

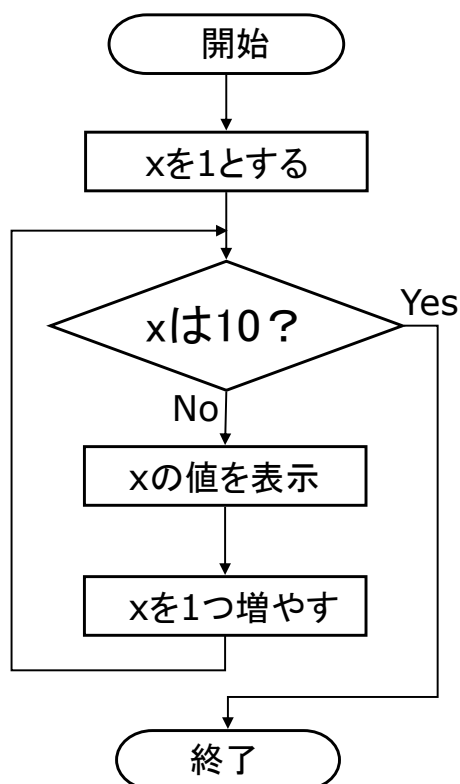
たとえば...

```
if (今日 は 平日)
{
    学校へ行く...;
}
```

2. 条件分岐

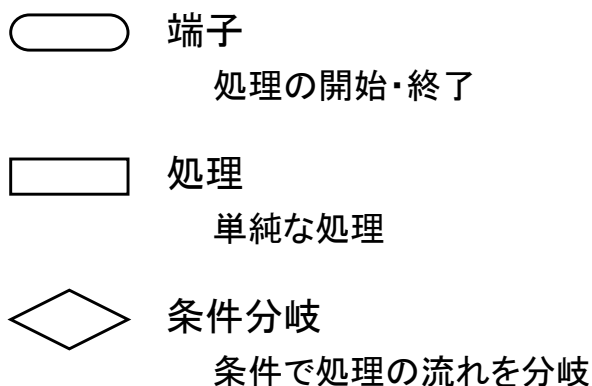
5

流れ図/Flow Chart



様々な処理の流れを図示するときの書式
(プログラムに限らず)

自分の作ったプログラムを理論的に記述する際
(レポート・論文など)に利用する



最低限、この3種類だけで図示できる

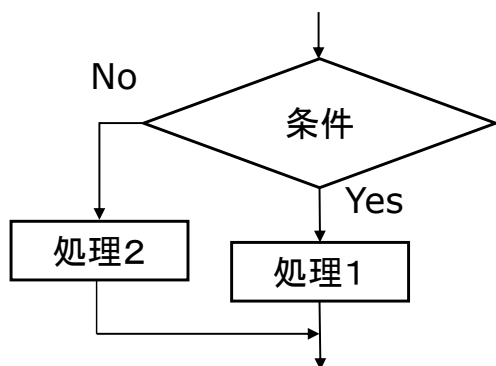
2. 条件分岐

6

if～else文

条件を満たさなかった場合の処理も用意できる

流れ図/Flow Chart



```
if (条件文)
{
    処理1;
}else{
    処理2;
}
```

たとえば...

```
if (今日は 平日) {
    学校へ行く...;
}else{
    午後まで寝る!;
}
```

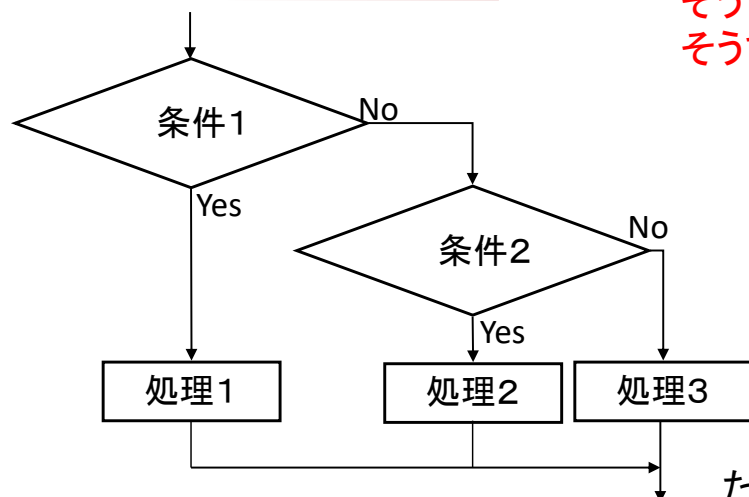
2. 条件分岐

7

if～else if文

2番目以降の条件を提示する

流れ図/Flow Chart



条件1を満たすなら処理1を実行、
そうでなく条件2を満たすなら処理2を実行、
それでもなければ処理3を実行

```
if (条件1) {  
    処理1;  
} else if (条件2) {  
    処理2;  
} else {  
    処理3;  
}
```

たとえば・・・

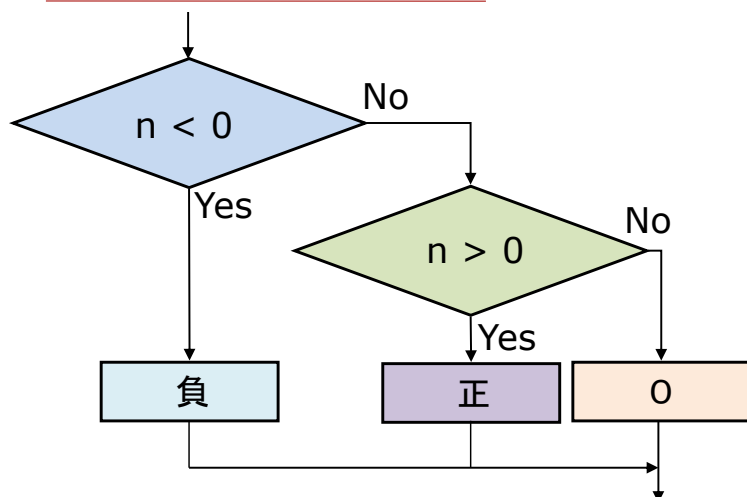
```
if (今日 は 平日) {学校へ行く・・・;}  
else if(今 は 春) {花見に行く;}  
else {午後まで寝る!;}  
}
```

2. 条件分岐

8

例題: 入力した値の正負を表示するプログラム

流れ図/Flow Chart



```
#include<stdio.h>  
int main(void) {  
    int n;  
    printf("nを入力:");  
    scanf("%d", &n);  
  
    if (n < 0) {  
        printf("負です。¥n");  
    } else if (n > 0) {  
        printf("正です。¥n");  
    } else {  
        printf("0です。¥n");  
    }  
    return 0;  
}
```

例題: 入力した値の正負を表示するプログラム

関係演算子

条件文では様々な「関係演算子」が用いられます

$a == b$: aとbが等しいか？
 $a != b$: aとbが等しくないか？
 $a > b$: bよりもaが大きいか？
 $a < b$: aよりもbが大きいか？
 $a \geq b$: aがb以上か？
 $a \leq b$: bがa以上か？

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n < 0) {
        printf("負です。¥n");
    } else if (n > 0) {
        printf("正です。¥n");
    } else {
        printf("0です。¥n");
    }
    return 0;
}
```

2. 条件分岐

例題: 条件の組み合わせ

入力した値が3と8の公倍数かどうかを調べる

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n%3 == 0 && n%8 == 0)
    {
        printf("3と8の公倍数です。¥n");
    }
    return 0;
}
```

論理演算子

- 「条件1 && 条件2」
条件1と条件2が両方とも成り立っている場合に、「真」となる
- 「条件1 || 条件2」
条件1と条件2のうちどちらかが成り立っている場合に、「真」となる
- 「!(条件1)」
条件1の真偽を逆にする

ここから演習課題です

5. 演習課題

12

演習3-1 if文の実行

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n < 0) {
        printf("負です。¥n");
    }
    return 0;
}
```

この通りプログラムを書いて、実行してみよう
nの値によって出力が変わることを確認しよう

5. 演習課題

13

演習3-2 if～else文の実行

この通りプログラムを書いて、実行してみよう
nの値によって出力が変わることを確認しよう
条件やprintf文内も変えてみよう

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n < 0) {
        printf("負です。¥n");
    } else {
        printf("負ではありません。¥n");
    }
    return 0;
}
```

5. 演習課題

14

演習3-3 偶数・奇数の判定

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if ( ) {
        printf("%dは偶数です。¥n", n);
    } else {
        printf("%dは奇数です。¥n", n);
    }
    return 0;
}
```

正しい条件文を埋めて
実行してみよう

演習3-4 論理演算子の利用

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n%3 == 0 && n%8 == 0)
    {
        printf("3と8の公倍数です。¥n");
    }
    return 0;
}
```

この通りプログラムを書いて、
実行してみよう
論理演算子を使って、様々な
条件文を作成してみよう

演習3-5 うるう年判定

年数を入力(例:2023)し、その年がうるう年かどうか
判定するプログラムを作成せよ。

うるう年:

1. 400で割り切れる年は、うるう年
2. 400で割り切れず、100で割り切れる年は、うるう年ではない
3. 400でも100でも割り切れない場合、4で割り切れる年は、
うるう年、そうでない場合は、うるう年ではない

例:

- ・2016年 → うるう年
- ・2015年 → うるう年ではない
- ・1900年 → うるう年ではない
- ・2000年 → うるう年

if～else文を複数使うと
実現できます。
条件分岐の**順番**が大事！

5. 演習課題 (※発展課題の提出は必須ではありません) 17

発展3-1 関数の共有点の個数

a, b, c の3つの係数を入力し、関数 $y = ax^2 + bx + c$ と x 軸 ($y = 0$) との共有点の個数を調べるプログラムを作成せよ。例外処理(例: $a = 0$)も考慮してみよ。

二次方程式 $ax^2 + bx + c = 0$ の判別式 $D = b^2 - 4ac$ を利用する:

- $D > 0 \rightarrow$ 共有点2個
- $D = 0 \rightarrow$ 共有点1個
- $D < 0 \rightarrow$ 共有点0個

例外処理:

係数としてどんな値が入力された場合でも、最後まで実行できるようにすることが重要。

補足 int main(void)と C言語標準規格について 18

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    return 0;
```

```
}
```

Q. main関数はなぜint型なのか？

引数はなぜvoid型なのか？また、void以外にどのような型を取るのか
return 0; は何者か

A. 下記のような標準規格で定められた仕様・フォーマットに従って
できることが決まっています(教科書6.5節でも説明予定)

void main(void)など試してもエラーが出るだけで壊れたりもしないので
積極的に試してください

C言語仕様を定めるISO標準規格(C99)の日本語版規格より抜粋

5.1.2.2.1 プログラム開始処理 プログラム開始処理において呼び出される関数の名前は、main とする。
処理系は、この関数に対して関数原型を宣言しない。この関数は、次の4種類の方法のいずれかで定義しなければならない。

– 返却値の型 int をもち仮引数をもたない関数

```
int main(void) { /* ... */ }
```

– 二つの仮引数をもつ関数 (仮引数は、これらが宣言された関数に対して局所的であるため、どのような名前を使用してもよいが、ここでは argc 及び argv とする。)

```
int main(int argc, char * argv[]) { /* ... */ }
```

– 上に掲げた二つの方法のいずれかと等価な方法()

– 上に掲げた三つの方法のいずれでもない処理系定義の方法

二つの仮引数を定義する場合、関数 main の仮引数は、次の制約に従わなければならない。

– argc の値は、非負でなければならない。

続き: <https://kikakurui.com/x3/X3010-2003-01.html>

※1 JIS規格・Japan Industrial Standard

※2 関数原型宣言＝関数プロトタイプ宣言
有用な宣言ですが、しばらくは不要です
第6回演習や教科書10章などで扱います

本日の説明内容はおしまいです。
課題に取り組んでください。